

Other questions

1. ①. HT, m pos, open addr, n elems. Can $m > n$?

In open addressing, all elems are stored directly in the HT.
Each position stores at most 1 elem $\Rightarrow m \leq n$

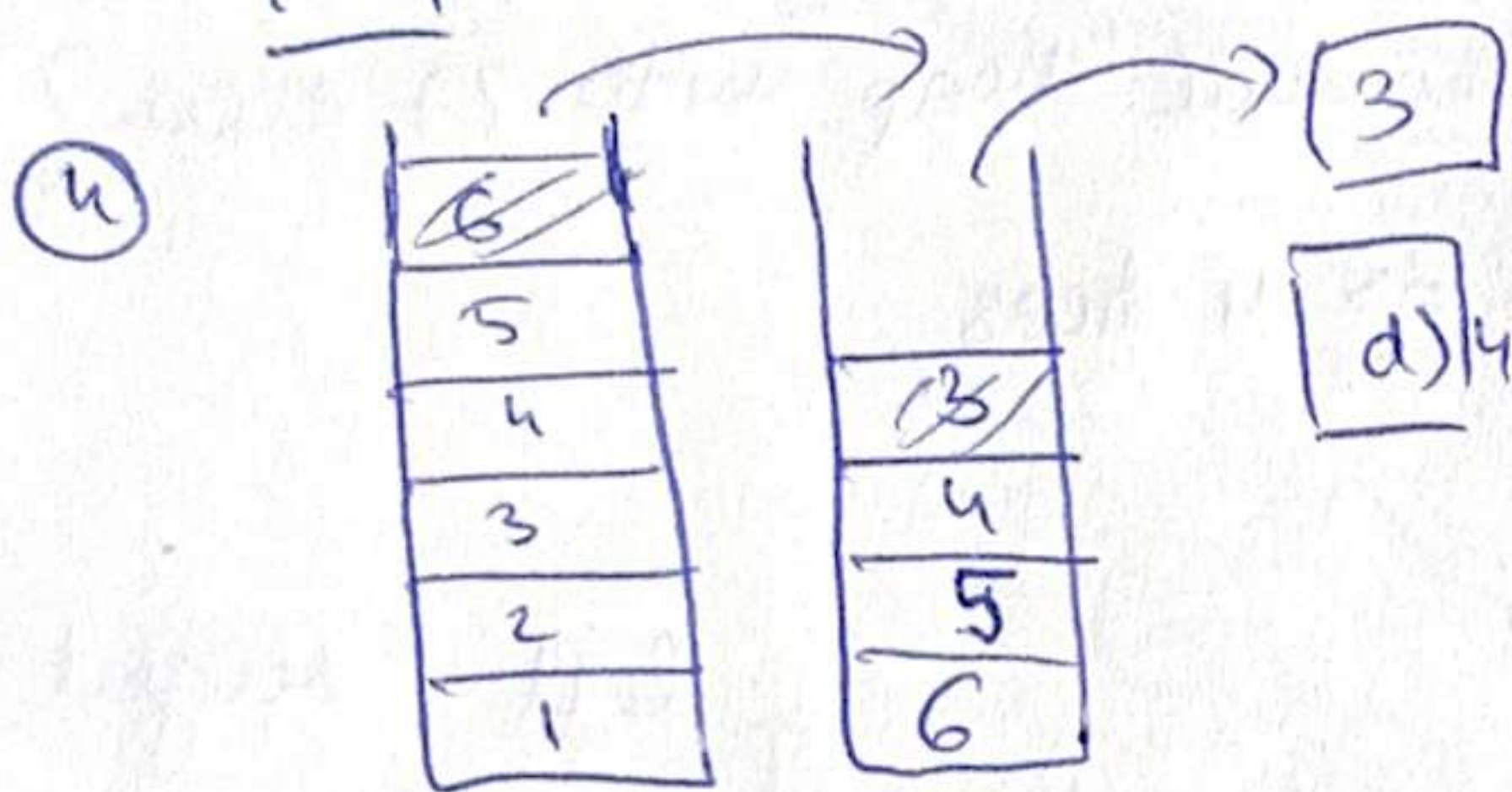
[a] Not possible, no matter how the h is defined.

② HT, m pos, open addr, n elems. Search e ; more than m pos checked?

[b] only if $\exists e$.

③. BST. instead of λ ?

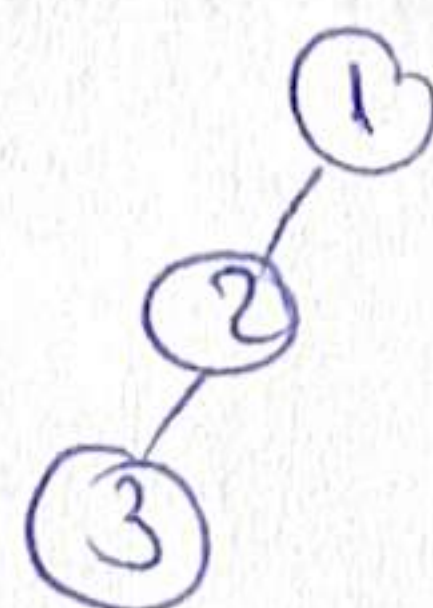
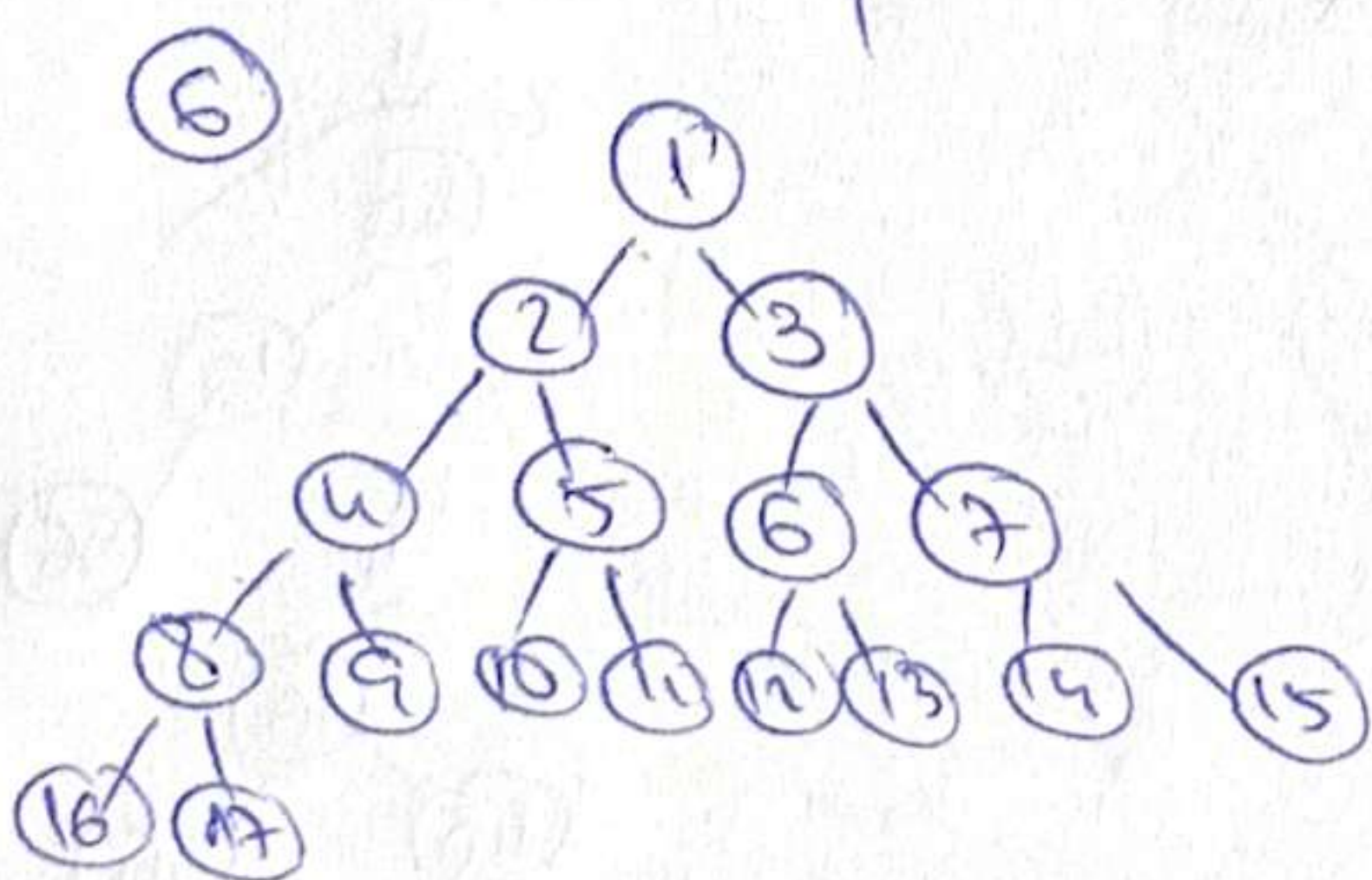
[d] 26



⑤. $467 + 1 - 25 * + +$

Token	Stack
4	4
6	46
7	467
+	4, 13
1	4, 13, 1
-	4, 12
2	4, 12, 2
5	4, 12, 2, 5
*	4, 12, 10
+	4, 22
+	26

[e] > 15



$$H_{max} = 20 - 1 = 19$$

$$H_{min} = 4$$

$$\Rightarrow H_{max} - H_{min} = 19 - 4 = 15 \quad [b]$$

⑦. DS for list if we frequently getElement() from pos p?
(best)

a) dynarray

b), c) $\rightarrow$ su, dl $\rightarrow$ need traversals $\Theta(n)$

d) $\rightarrow$ skip list $\rightarrow$ not designed for positions

e) Bin. heap $\rightarrow$ not a list

f) HT $\rightarrow$ no pos.

⑧ $12m^3 + m \log_2 m + 52$

a) $O(m^4)$

d) $\Omega(m^3)$

e) $\Omega(1)$

⑨ How many binomial trees in a binomial heap with 27 elem?

$$27 = 2^4 + 2^3 + 2^1 + 2^0 \Rightarrow 0, 1, 3, 4 \Rightarrow 4 \text{ trees}$$

⑩ preorder: root-left-right

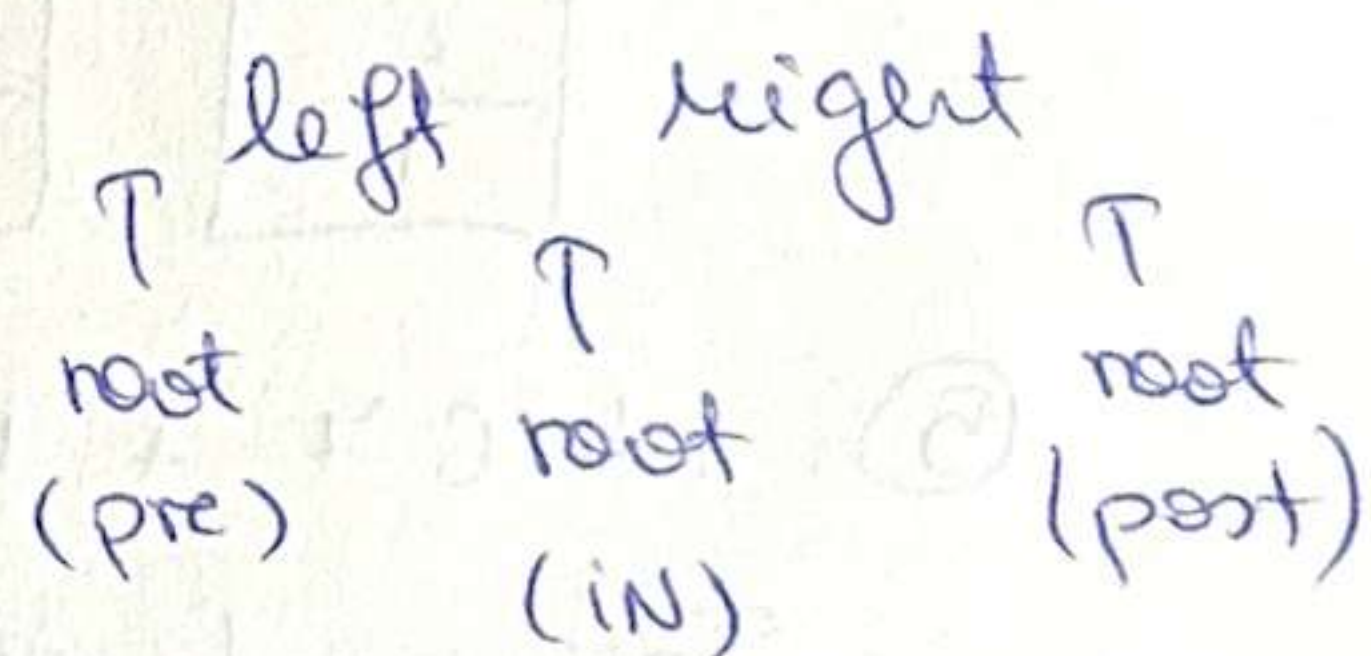
~~DE~~ DABCEHFGNI

! postorder: left-right-root

~~A~~ ECB AFNIGH D

inorder: left-root-right

BECA D F H N G I

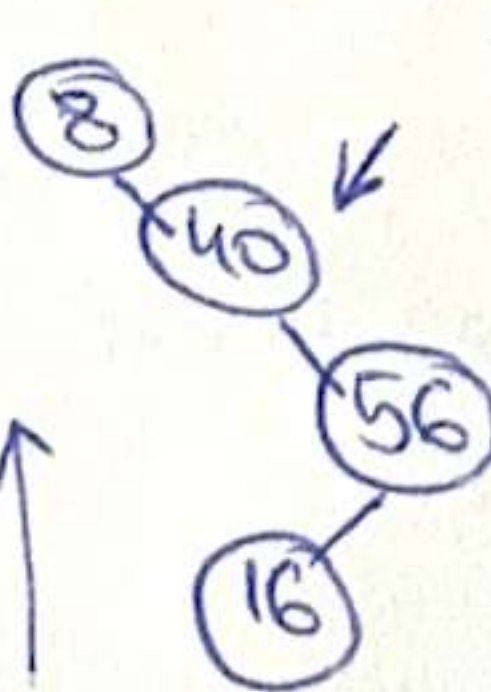
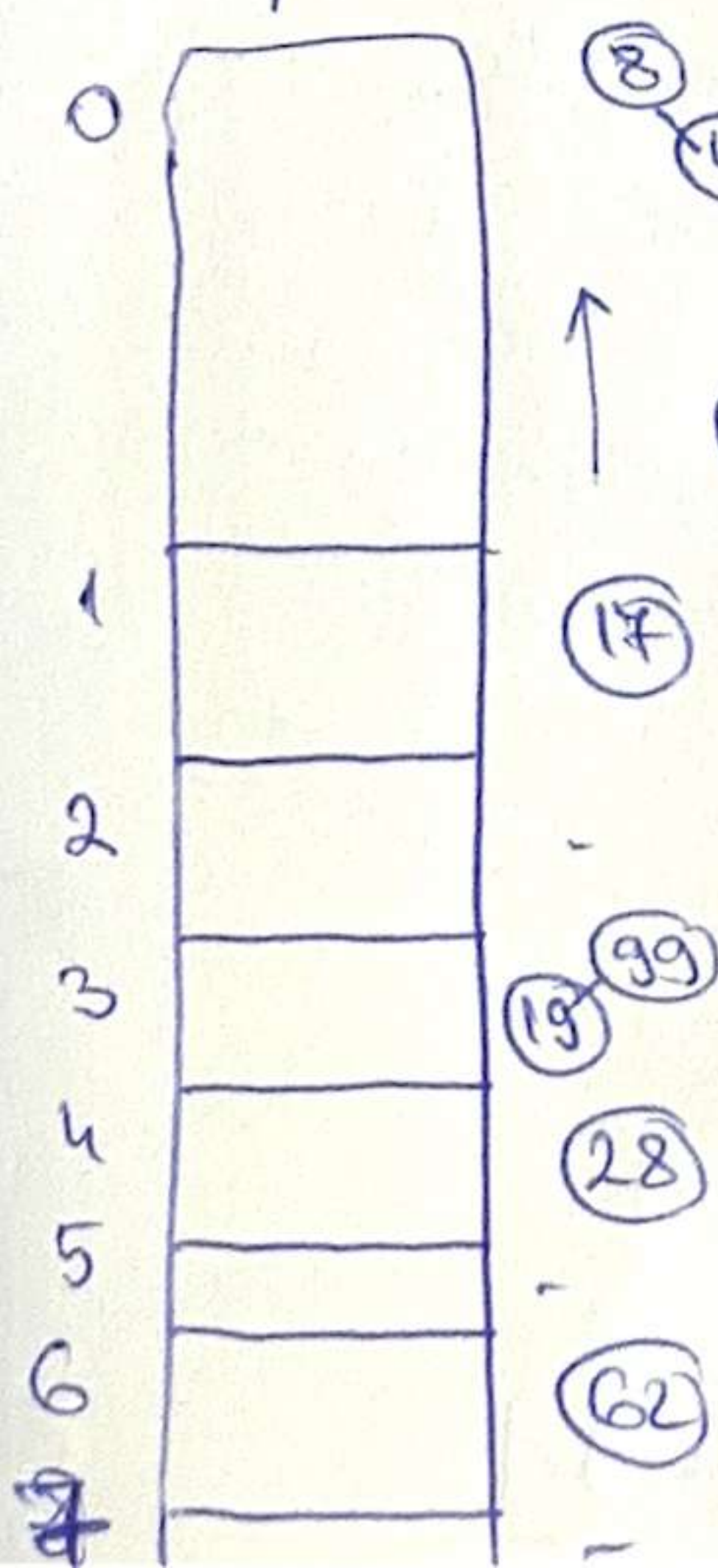


⑪ HT, $m=8$, sep ch, div method, root AVL. (instead of a LL).

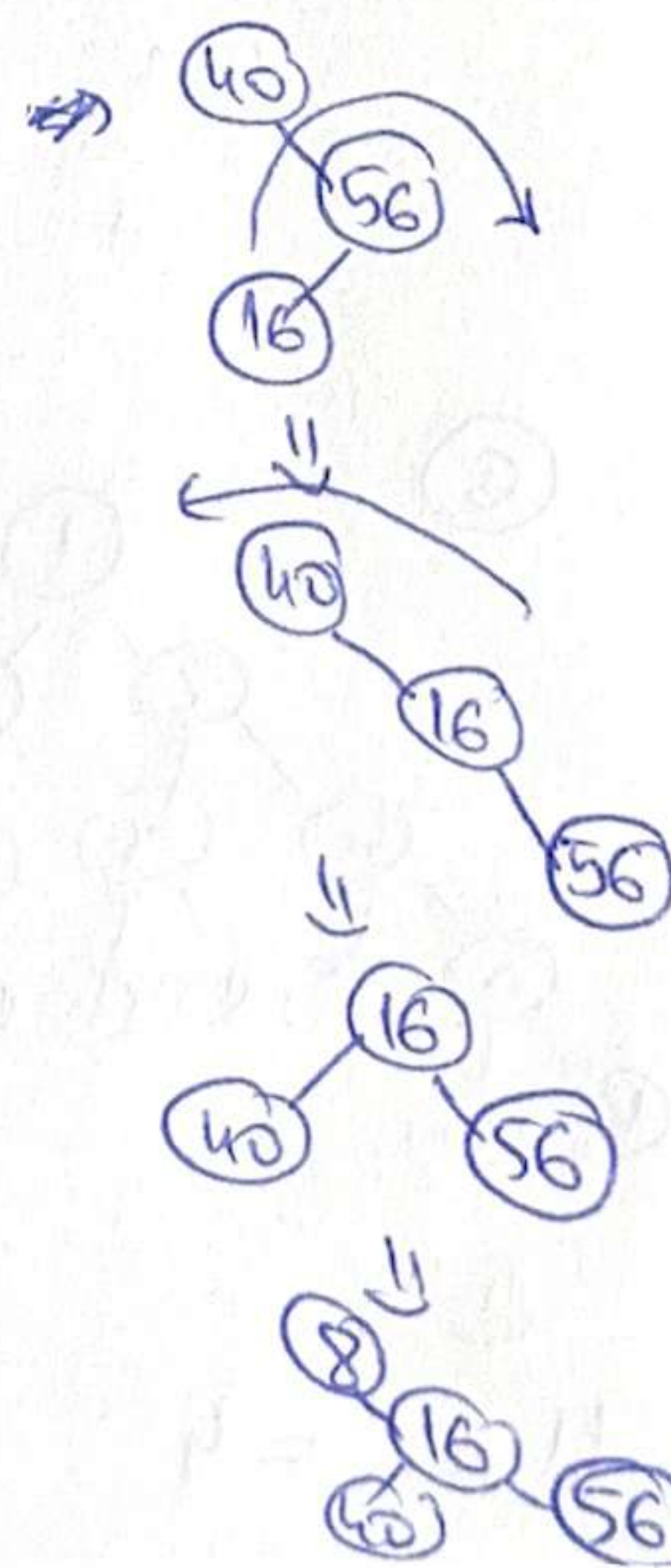
add 8, 99, 40, ...

$$\alpha = \frac{n}{m} = \frac{9}{8} \text{ load factor}$$

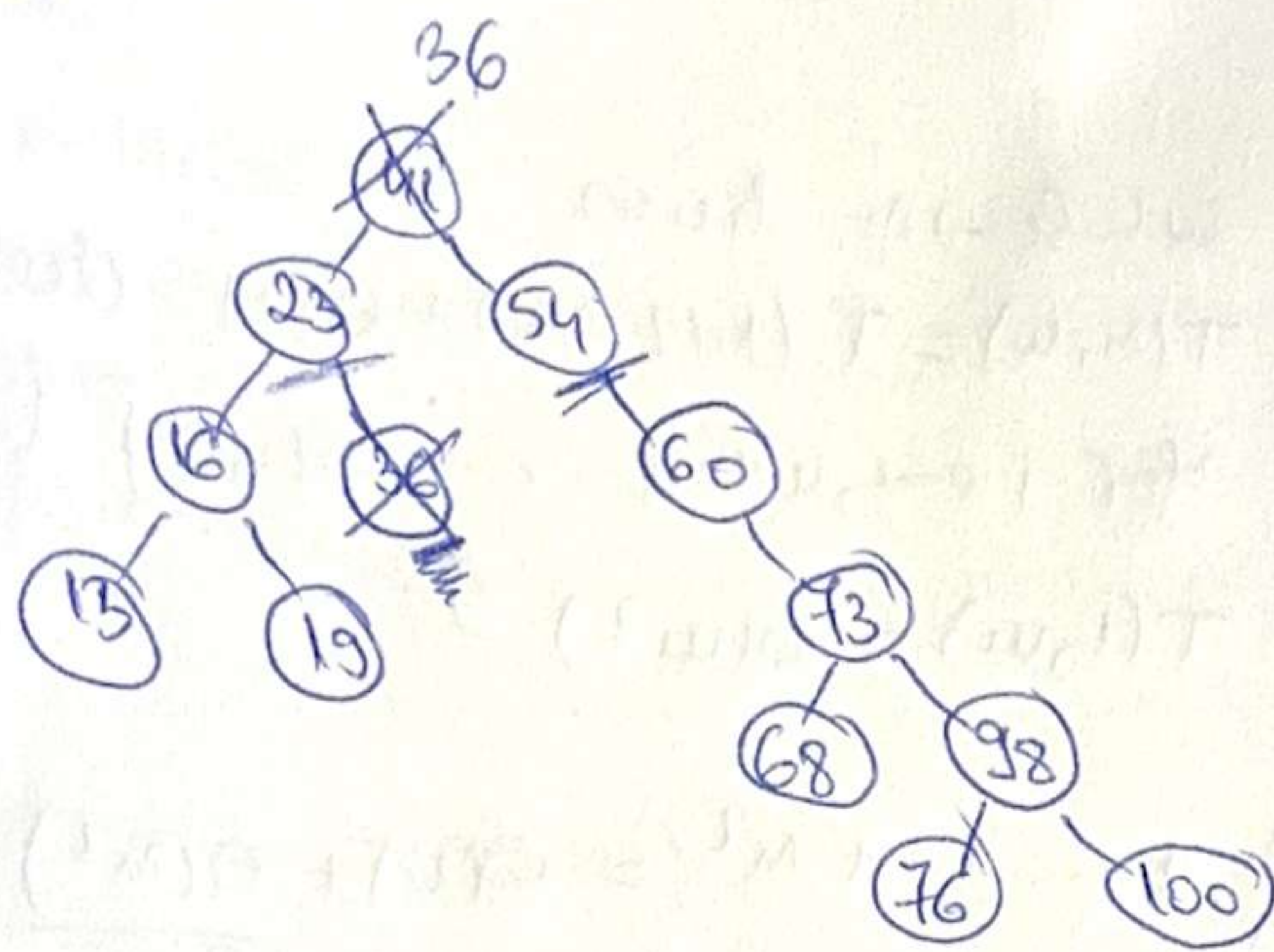
e	8	99	40	19	56	16	17	28	62
h(e)	0	3	0	3	0	0	1	4	6



unbalanced,
BF(8) = 0 - 3 = -3 $\Rightarrow$ heavy-right
BF(40) = 0 - 2 = -2 $\Rightarrow$ balanced, heavy-right ? RL
BF(56) = 1 - 0 = 1 $\Rightarrow$ balanced, heavy-left
BF(16) = 0 - 0 = 0 $\Rightarrow$ balanced.

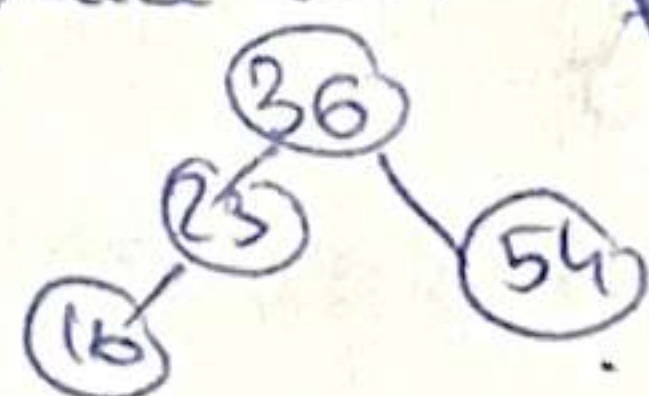


⑩ ≤ BST

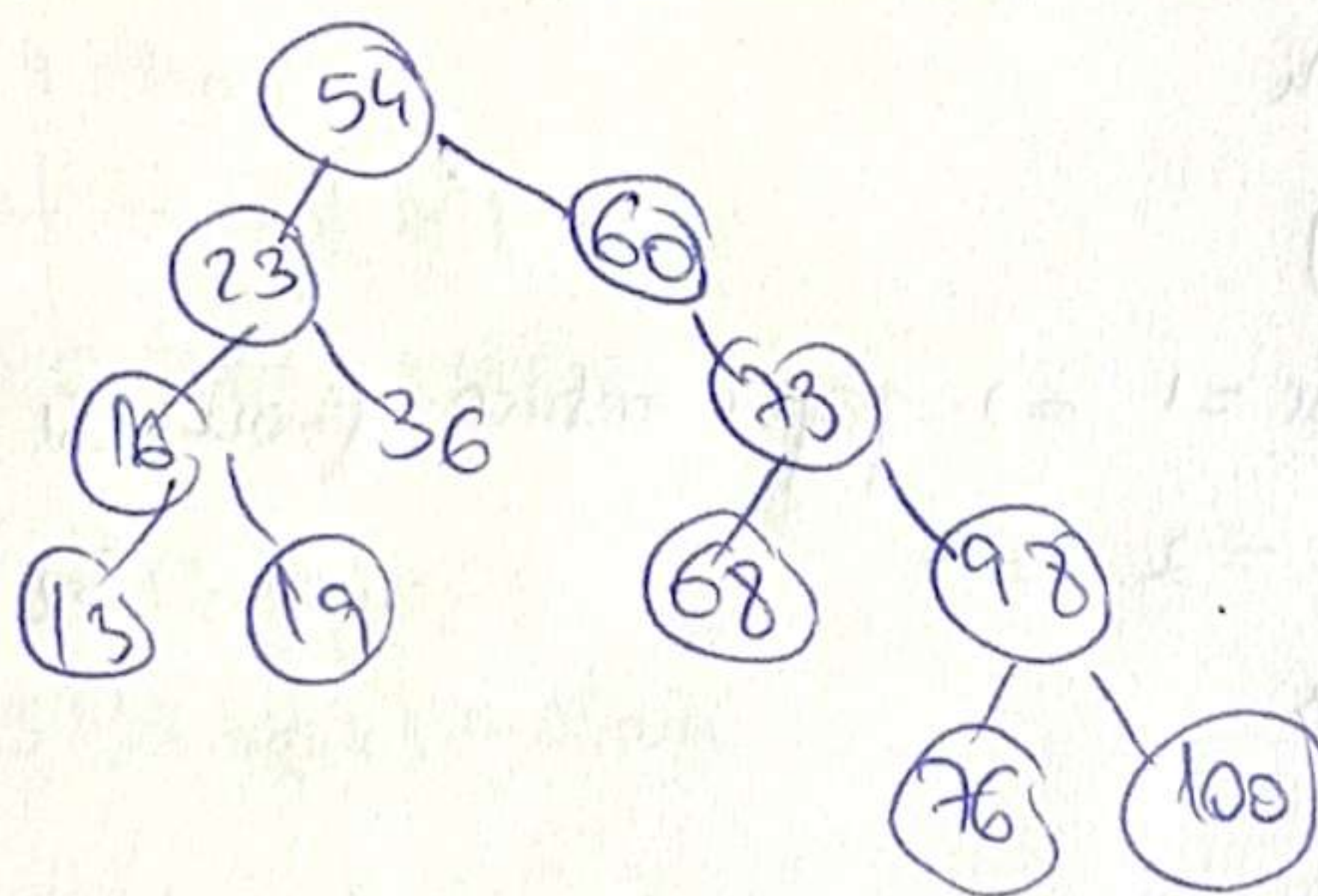


remove 41 ⇒ replace with predecessor (max value in its left subtree)

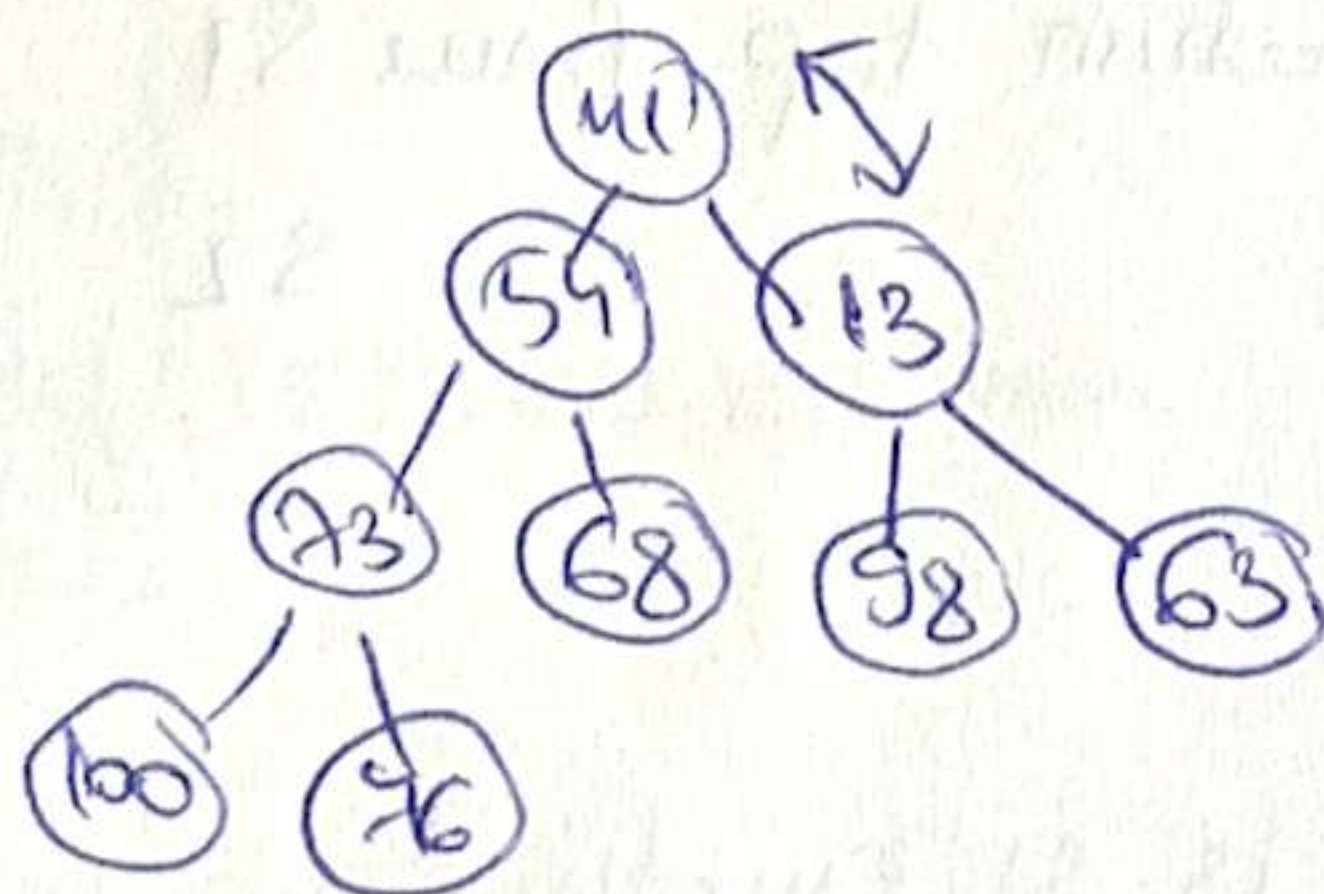
36.



⇒ replace with successor (min value in its right subtree)

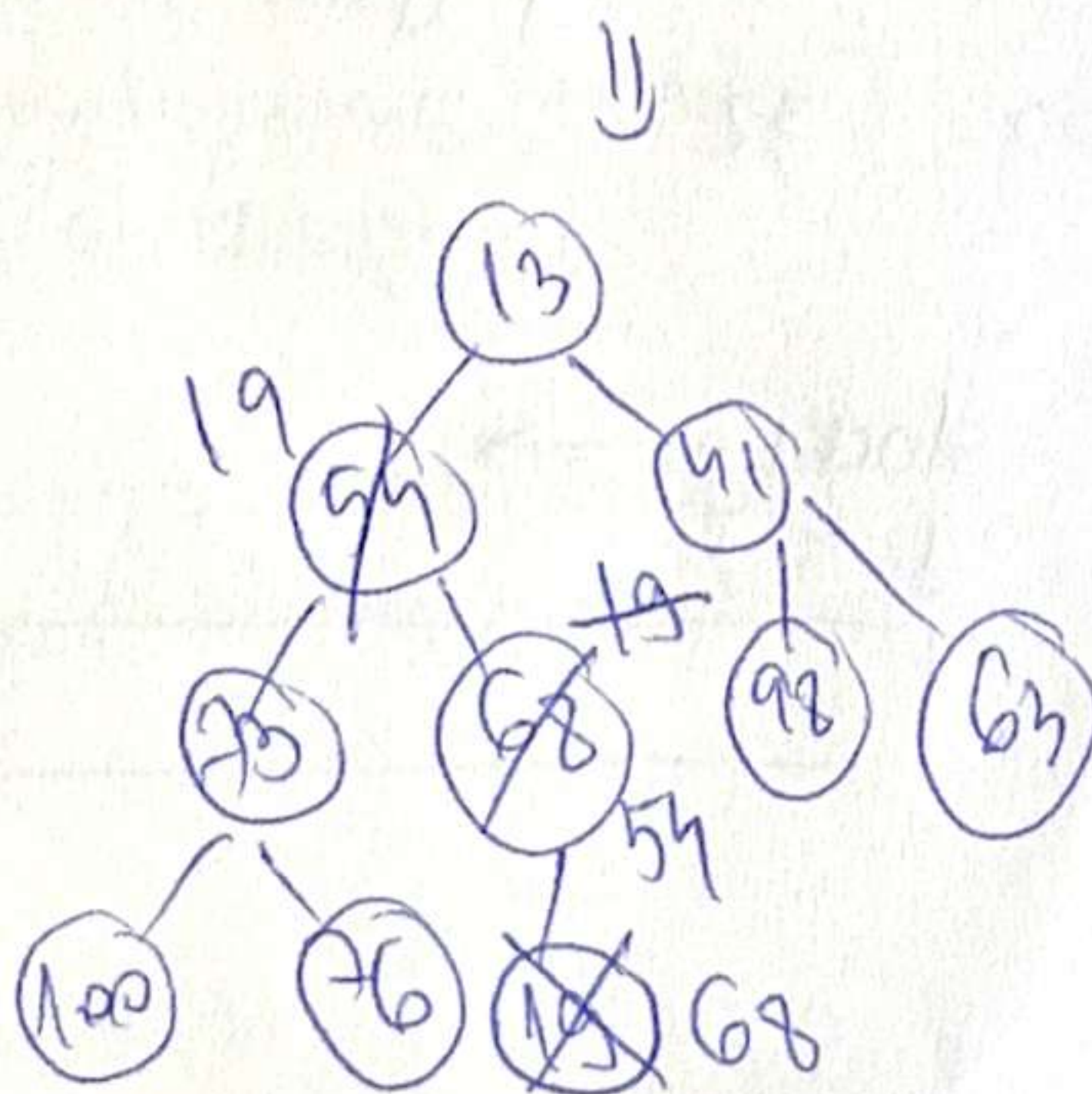


⑪ Bin. heap?

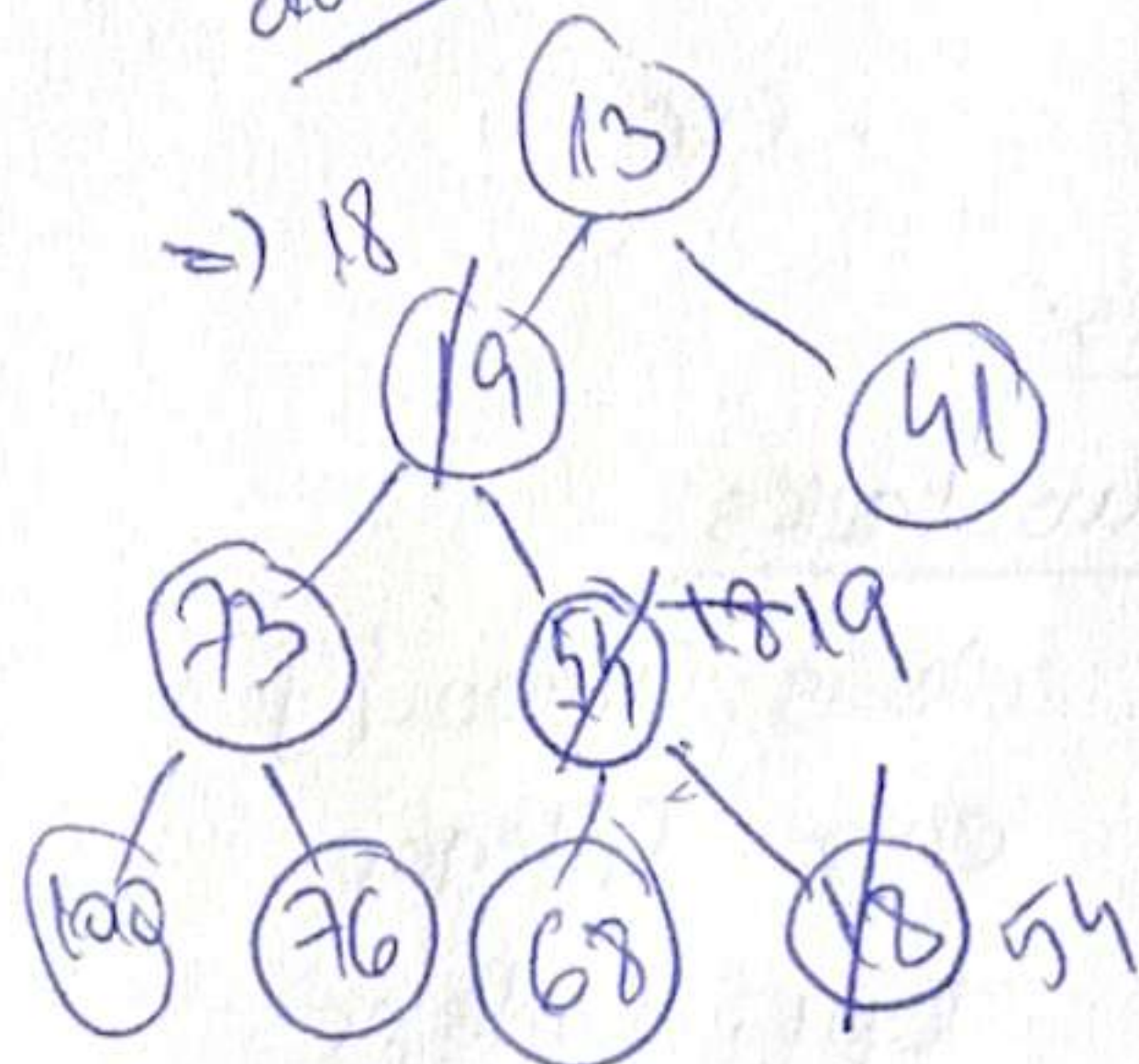


not a ~~max~~ heap

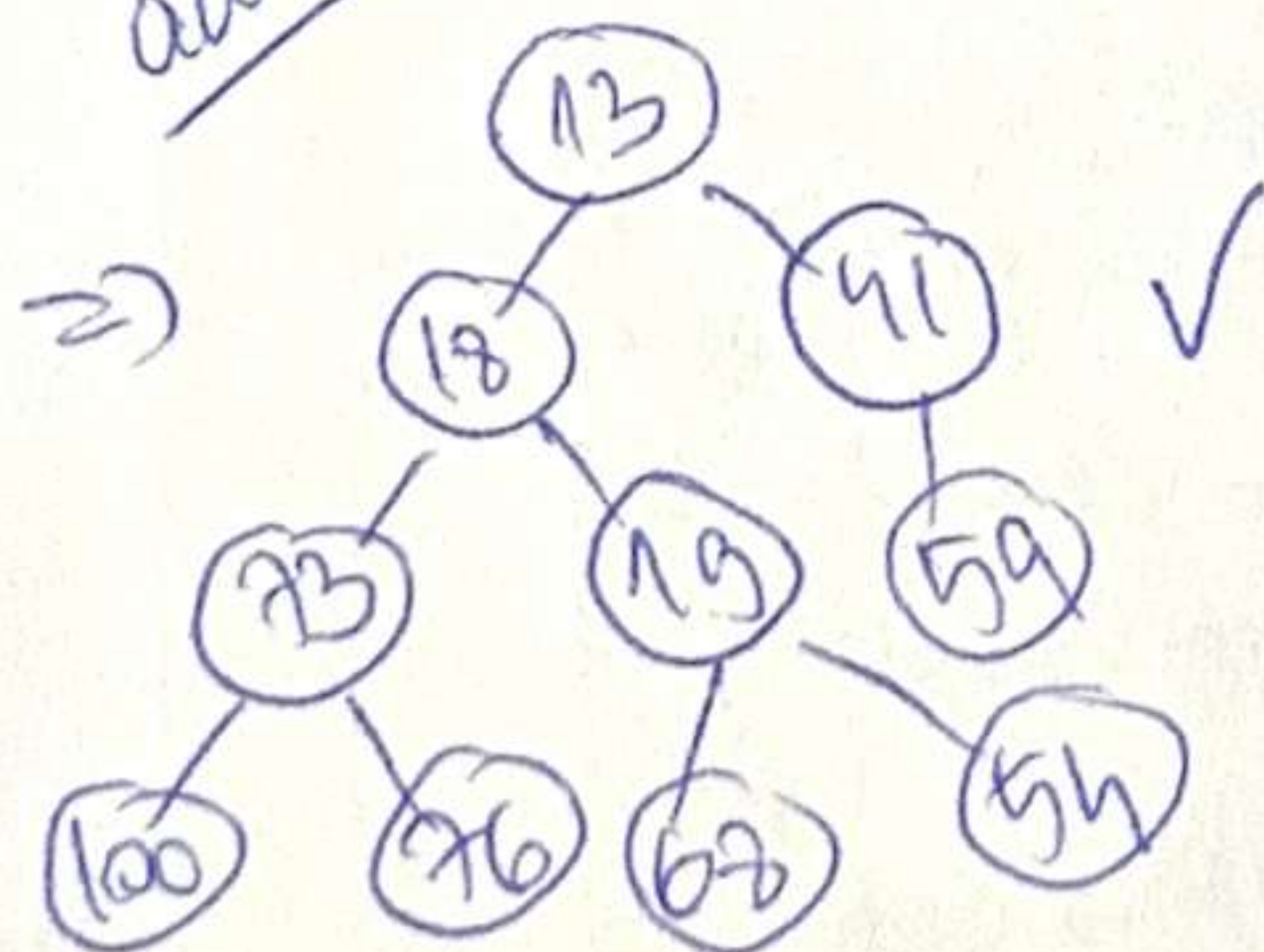
add 19



add 18



add 59



⑫

op(m, n)

k = m/2

i = 3

i = 3 + n

n = n - 1

while → m times

$T(m, n) = T(m/2, n) + \Theta(m)$ (recursive)

for $i \leftarrow 1, n \times n \leftarrow \Theta(n^2)$ base case

$T(1, n) = \Theta(n^2) \uparrow$

$$T(m, n) = m + \frac{m}{2} + \frac{m}{4} + \dots + 1 + n^2 = \Theta(m) + \Theta(n^2) = \Theta(n^2)$$

⑬ ADT TwoStacks : $\Theta(1)$ complexity

push(ts, elem, stackNr)

• stackNr = 1 ⇒ push to S1

• = 2 ⇒ S2

• exception

pop(ts, stackNr)

• stackNr = 1 ⇒ pop + return from S1

• = 2 ⇒ S2

• exception

top(ts, stackNr)

• stackNr = 1 ⇒ return top from S1

• = 2 ⇒ S2

• exception

isEmpty(ts, stackNr)

• stackNr = 1 ⇒ is S1 empty?

• = 2 ⇒ S2

• exc.

Repres:

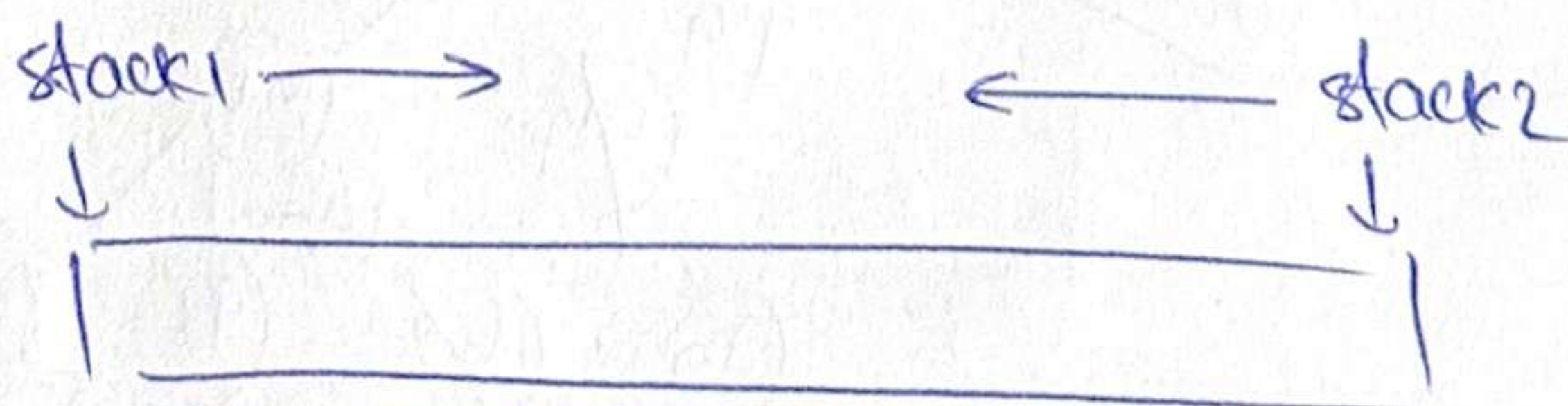
TwoStacks:

elems: TElem[]

cap: Integer

top1: Integer

top2: Integer



of $\text{init}(ts)$ is

```
ts.cap  $\leftarrow$  initialCap  
ts.elems  $\leftarrow$  allocate(ts.cap)  
ts.top1  $\leftarrow$  -1  
ts.top2  $\leftarrow$  ts.cap
```

$\text{push}(ts, \text{elem}, \text{stackNr})$

```
if stackNr  $\neq$  1 and stackNr  $\neq$  2 then  
  @throw ...
```

```
if ts.top1  $\neq$  ts.top2 then  
  resize(ts)
```

```
if stackNr = 1 then
```

```
  ts.top1  $\leftarrow$  ts.top1 + 1
```

```
  ts.elems[ts.top1]  $\leftarrow$  elem
```

```
else
```

```
  ts.top2  $\leftarrow$  ts.top2 - 1
```

```
  ts.elems[ts.top2]  $\leftarrow$  elem
```

$\text{pop}(ts, \text{stackNr})$

```
if isEmpty(ts, stackNr) then  
  @throw ...
```

```
if stackNr = 1 then
```

```
  elem  $\leftarrow$  ts.elems[ts.top1]
```

```
  ts.top1  $\leftarrow$  ts.top1 - 1
```

```
else
```

```
  elem  $\leftarrow$  ts.elems[ts.top2]
```

```
  ts.top2  $\leftarrow$  ts.top2 + 1
```

$\text{top}(ts, \text{stackNr})$

```
if isEmpty(ts, stackNr) then  
  @throw
```

```
if stackNr = 1 then
```

```
  top  $\leftarrow$  ts.elems[ts.top1]
```

```
else
```

```
  top  $\leftarrow$  ts.elems[ts.top2]
```


function isEmpty (ts, stackNr)

```

if stackNr = 1 then
    isEmpty ← ts.top1 = -1
else if stackNr = 2 then
    isEmpty ← ts.top2 = ts.cop
else
    @ throw

```

$\Theta(1)$ because usually we only change one index and assign one value and resize is rare.

WC: $\Theta(n)$ (resize)
amortized: $\Theta(1)$

(14). Find max value in BT of integers

BT:

root: $\uparrow$ BTNode

Node:

info: Int
left: $\uparrow$ Node
right: $\uparrow$ Node

function max(^{bt}~~node~~)

```

maxValue ← -1
traverse (bt.root, maxValue)
max ← maxValue

```

subalg traverse (node, maxValue)

```

if node ≠ Nil then
    if [node].info > maxValue then
        maxValue ← [node].info
    traverse ([node].left, maxValue)
    traverse ([node].right, maxValue)

```

$\Theta(n)$ visit all nodes

(15) @Quarrier -